HSLU

School of Computer Science and Information Technology

Lucerne University of Applied Sciences and Arts (Switzerland)

Predicting Apartment Rental Prices in Switzerland

A Supervised Machine Learning Approach
for the Swiss Cold-Rent Market

DSPRO1 Final Report, Spring 2026 (FS26)

presented to the School of Computer Science and Information Technology of
Lucerne University of Applied Sciences and Arts

by

Elias Martinelli | Timo Schlumpf

Team 8

Supervisor: Elena Nazarenko

Repository: `github.com/wadafacc/dspro1`

Date: May 26, 2026

Contents

Abstract

This project predicts monthly cold rent for Swiss apartment listings. We start from a self-scraped `rentumo.ch` snapshot (∼4,500 modelling rows, collected on 13 April 2026) and enrich it with the Federal Register of Buildings and Dwellings (GWR) and several swisstopo layers, including public-transport accessibility, elevation, solar suitability and hectare-grid population.

The final decision is not based on the lowest RMSE alone. LightGBM is the strongest raw-accuracy benchmark and reaches about 393 CHF RMSE after adding KNN-distance features, but it also shows a much larger train/eval gap. We therefore use **GradientBoosting with the `ALL+geo` feature set** as the final model. It is slightly less accurate on the evaluation split (around 425 CHF RMSE in the main comparison), but its overfitting gap is much smaller, at roughly 60 CHF. In practical terms, it is the model we trust more outside the exact training sample. The tree-based hypothesis is supported clearly: all tree-based models outperform the linear Ridge benchmark. The main remaining weakness is the expensive upper end of the market, where missing qualitative signals such as view, floor, renovation state and finish become decisive.

1 Introduction

Apartment rents in Switzerland differ strongly by location, building type, and apartment characteristics. For someone comparing a listing such as "CHF 2'400 per month, 3 rooms, 80 m^2, Zurich", it is difficult to judge whether the advertised price is reasonable. Public rent indicators are useful for understanding general market trends, but they do not give a concrete estimate for one specific apartment.

This project builds a supervised regression model for predicting the monthly cold rent (*Kaltmiete*, in CHF) of individual Swiss apartment listings. The underlying dataset is *not* a ready-made benchmark: we collected it ourselves by writing a custom scraper for `rentumo.ch` [13], a Swiss rental aggregator, and enriched the resulting listings with official Swiss open-register data from GWR and swisstopo. The task is treated as a tabular regression problem. Each apartment is described by structural features such as living area, number of rooms and year of construction, as well as location-based features such as coordinates, public-transport accessibility, elevation, population density and solar suitability. The target variable is the listed cold rent.

Research question and hypothesis. The main question is whether listing data, combined with Swiss open-register data from GWR and swisstopo, contains enough information to estimate apartment rents in a useful way. Based on the project proposal and the mid-term presentation, we formulated the following hypothesis:

> *"Location-related variables combined with structural apartment features contain sufficient signal to predict monthly rental prices in Switzerland with practically useful accuracy, and a Tree based model will outperform a Linear Regression model in doing so."*

We test this hypothesis with several levels of model complexity: a simple baseline that ignores all features, a linear Ridge model as an interpretable benchmark, and a family of tree-based models including RandomForest, GradientBoosting, XGBoost, LightGBM, and a Stacking ensemble. The hypothesis is therefore phrased at the model-*family* level rather than tying it to one specific algorithm, which reflects how the experimental design actually unfolded across the project.

Stakeholders and intended use. The main users are tenants who want a quick sanity check for an advertised rent. Landlords and property managers could also use the estimate as a market-based reference point. The model is only a decision-support tool: it is *not* a legal rent assessment and does not replace formal rent reviews under Swiss tenancy law (OR Art. 269 ff.). The Streamlit app in Section 5.11 shows how the trained `RentPredictor` can be used by a non-technical person, but the scientific contribution remains the data pipeline and the model comparison.

Report structure. The report is structured as follows. Section 2 gives an overview of related work in real-estate price prediction. Section 3 describes the data sources, the enrichment process

and the main quality checks. Section 4 explains the modelling pipeline, evaluation strategy and feature engineering. Section 5 presents the results, including model comparison, residual diagnostics and error analysis. Section 6 closes with the main conclusions, limitations and possible next steps.

2 State of the Art

Real-estate and rental-price prediction has been studied for a long time in both economics and machine learning. The traditional approach is *hedonic pricing*: prices are modelled as a function of structural and location-related characteristics, often using linear regression. This idea goes back to Rosen [14]. Hedonic models are still widely used, especially by statistical offices, but they require strong assumptions about the functional form and can struggle with complex non-linear effects.

Modern machine-learning approaches use more flexible models instead of manually specifying all relationships. Tree-based ensembles such as RandomForest [5] and gradient boosting [7] are particularly well suited for tabular regression. They can capture non-linear interactions, handle features with different scales and usually require only limited preprocessing. Optimised implementations such as XGBoost [6] and LightGBM [10] add efficient training and regularisation, and often perform strongly on tabular benchmarks [15].

Because rent estimates may influence real decisions, interpretability is important. SHAP values provide feature-attribution explanations for individual predictions and are commonly used together with tree-based models [11]. For documenting the dataset and model, we follow the ideas of *Datasheets for Datasets* [8] and *Model Cards* [12].

For the Swiss housing market specifically, two public data sources are critical: the Federal Register of Buildings and Dwellings (GWR), maintained by the Federal Statistical Office (BFS) [3], and the swisstopo geo-services exposed through GeoAdmin [9]. The GWR provides per-building attributes (construction year, number of dwellings, building footprint) indexed by EGID, while swisstopo provides geo-layer attributes such as public-transport accessibility [1], solar-suitability classes from the Federal Office of Energy (BFE) [2], and the BFS population-by-hectare grid [4].

3 Data

3.1 Sources and Collection

The primary dataset, *Rentumo Rental Listings Switzerland v1.0*, was collected from `rentumo.ch` in a single scrape on 13 April 2026. The scraper identified 25 002 entries on the platform, of which 9 994 listings could be extracted successfully. Many of the remaining pages were empty or non-functional placeholder pages, which appears to be a characteristic of the platform rather than an error in the scraper.

The scraper is implemented as a custom Rust library, `ScrapeGoat`, and operates in two sequential phases.

Phase 1: Listing index pages. The scraper issues requests to 750 paginated listing pages of the form `rentumo.ch/mietobjekte?types=apartment&page={1..750}`. Each page is parsed with CSS selectors targeting the `#listings` container, from which per-listing elements yield the number of rooms, the living area in square metres, the cold rent, and a unique URL slug (e.g. `/inserate/<slug>`). Listings for which any of these four fields cannot be parsed are silently dropped at this stage. Successfully extracted records are written to the `listings` table in the PostgreSQL database with an `ON CONFLICT (slug) DO NOTHING` guard, making the phase idempotent and safe to re-run.

Phase 2: Detail pages. All slugs stored in `listings` are retrieved from the database and used to construct individual detail URLs (`rentumo.ch/inserate/<slug>`). Each detail page is parsed to extract additional fields: the full address, the cold rent as confirmed on the detail page, the living area, the number of rooms, the auxiliary costs (*Nebenkosten*), the security deposit (*Kaution*), a free-text description from the JSON-LD *Apartment* schema block, and the earliest available move-in date (*Datum verfügbar*). These are written to a separate `listing_details` table with `ON CONFLICT (listing_id) DO UPDATE`, allowing the phase to be resumed or repeated without data loss.

Concurrency and politeness. Both phases are managed by a custom task orchestrator (`Mgt`) that maintains a fixed pool of up to 10 concurrent in-flight HTTP requests. A channel-based feedback loop ensures that a new request is dispatched only when an existing one completes, bounding peak load on the server. Each request carries a randomly sampled `User-Agent` header drawn from a pre-loaded list, and optional HTTP proxies can be configured via a plain-text file to distribute outbound traffic across multiple IP addresses.

Platform selection. We also considered `comparis.ch` as a possible data source, but decided not to use it. Comparis confirmed in writing that automated extraction would require formal authorisation and that access would likely be limited to around 300 records, which would not be sufficient for this modelling task. Rentumo was contacted by email before the scrape, but no response was received. The data is used only for the non-commercial academic purpose of the DSPRO1 module.

3.2 Enrichment

The scraped listings contain the address and a small set of structural attributes, including `area_sqm`, `rooms`, `price_cold` and `auxiliary_costs`. To make the dataset more useful for modelling, we enrich these listings in two steps:

1. **GWR (BFS).** Each listing address is geocoded and resolved to a building identifier (EGID) via the GeoAdmin SearchServer. Building-level attributes (`gbauj` / construction year, `ganzwhg` / number of dwellings, `garea` / footprint area) are then queried from the GWR record.

2. **swisstopo / GeoAdmin.** Using the LV95 coordinates (`lv95_east`, `lv95_north`), per-point attributes are queried from public layers: elevation (`elevation_m`, height service), public-transport accessibility class (`oev_score`, layer `ch.are.erreichbarkeit-oev`), solar suitability class 1–5 (`solar_class`, layer `ch.bfe.solarenergie-eignung-daecher`), and the population of the nearest hectare cell (`population`, BFS Volkszählung).

3.3 Storage and Schema

The raw and enriched data are stored in a PostgreSQL database on Neon Console. The relevant information is distributed across three tables: `listings` for the listing index, `listing_details` for the extracted apartment attributes, and `swisstopo_details` for the enriched geo-layer features. For modelling, these tables are joined into a flat file, `model.csv`, while requiring all mandatory columns to be present. After filtering and joining, the final modelling dataset contains approximately 4 500 rows with 12 columns: 11 predictors and `price_cold` as the target variable.

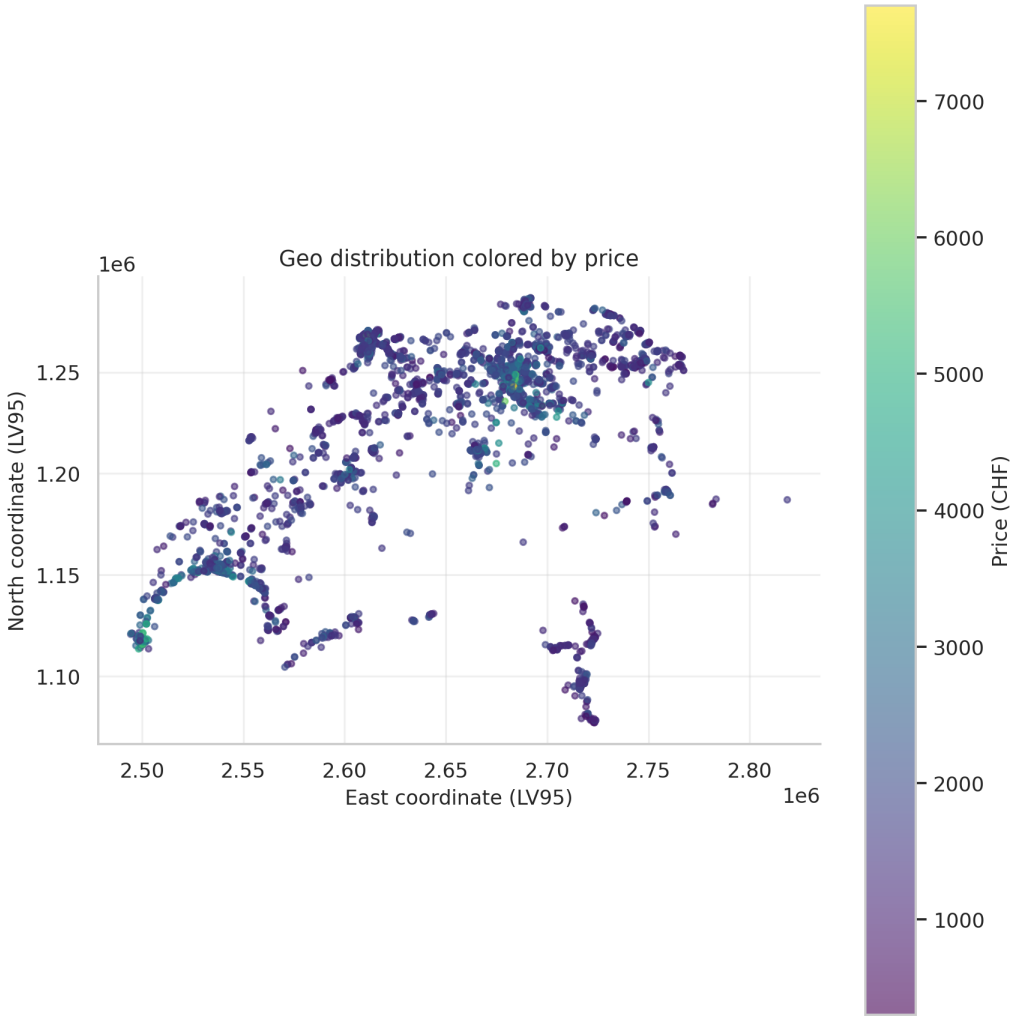

Figure 1: Geographic distribution of listings colored by cold rent.

3.4 Quality Assessment

- **Completeness.** Listings missing any of `address`, `price_cold`, `area_sqm`, or `rooms` are removed entirely. Listings marked "Preis auf Anfrage" are also removed. Of the 9 994 scraped listings, approximately 4 500 remain after the mandatory-field filter and the GWR/swisstopo join (cumulative loss is diagnosed in `final_records.ipynb`).
- **Accuracy.** An explicit outlier filter (`area_sqm` ≥ 10, `price_cold` ≥ 300) removes implausible listings. An `IsolationForest` diagnostic (Notebook Chapter 22.9) surfaces additional candidates for manual review.
- **Consistency.** Cantons are normalised to ISO 3166-2:CH (`ZH`, `BE`, ...). The construction year is stored as a four-digit integer; missing GWR values are median-imputed.
- **Timeliness.** The dataset is a single point-in-time snapshot from 13 April 2026. Temporal price changes after that date are not reflected; time-series modelling is explicitly out of scope.

The two most important data checks are also visible in the later modelling results: the target variable is strongly right-skewed, and the strongest numerical signal comes from apartment size, local price level and coordinates. This is why the report evaluates the model not only through global RMSE, but also through residual plots and price-band errors.

3.5 Legal and Ethical Considerations

No personal data is collected. Landlord names and contact details are explicitly excluded by the scraper. Only the address is retained and is used solely for geocoding and building-level enrichment. The project does not process any special-category data defined under the revised Swiss Federal Act on Data Protection (nFADP). The dataset is used exclusively for academic purposes and is not redistributed.

4 Methods

4.1 Pipeline Overview

The modelling pipeline is implemented in `src/notebooks/model_v3_clean.ipynb`. The notebook follows the same order as the analysis itself: clean the modelling table, build a baseline, compare model families, add location-aware features, diagnose overfitting, and then choose the model that generalises most consistently. The end-to-end prediction logic is wrapped in a `RentPredictor` Python class (Notebook Chapter 24.2), saved with `joblib`, and reused by the Streamlit app. The final saved model is `models/gradient_boosting_all_geo.joblib`: a `GradientBoostingRegressor` trained on the `ALL+geo` feature set.

4.2 Train/Eval/Test Split

We use a fixed random seed (`random_state=42`) to make the split reproducible. The main experiments use an 80% training set and a 20% evaluation set. For the final hold-out check (Notebook Chapter 24.6), we additionally use a 60/20/20 train/validation/test split. The evaluation and test sets are not used for model fitting. A Kolmogorov-Smirnov test (Chapter 22.13) is used to check that the random split does not introduce a strong distribution shift between training and evaluation data.

4.3 Feature Engineering

In addition to the raw features, three engineered features are added (Chapter 6): `building_age = REFERENCE_YEAR - year_built`, `area_per_room = area / rooms` using safe division, and `land_area_per_apartment = garea / ganzwhg`. Two further feature groups are created using only `train_df`, so that information from the evaluation set does not leak into training:

- **Geo-clusters.** `KMeans` on standardised LV95 coordinates produces a discrete `geo_cluster` feature (Chapter 21). The number of clusters is tuned on Eval RMSE.
- **KNN-distance features.** For each apartment, the median price of its 10 nearest neighbours in coordinate space (`knn_price_median`) and the corresponding mean (`knn_price_mean`) are added (Chapter 23.12). The nearest-neighbour index is fit on training data only; eval points query against the training index.

The final feature set is `ALL+geo`. It keeps the tabular signal from the apartment, building and location enrichments and adds the discrete `geo_cluster`. The geo-cluster is kept because Swiss rents are strongly spatial: even if it does not change every single prediction dramatically, it gives the model a compact way to represent local market segments.

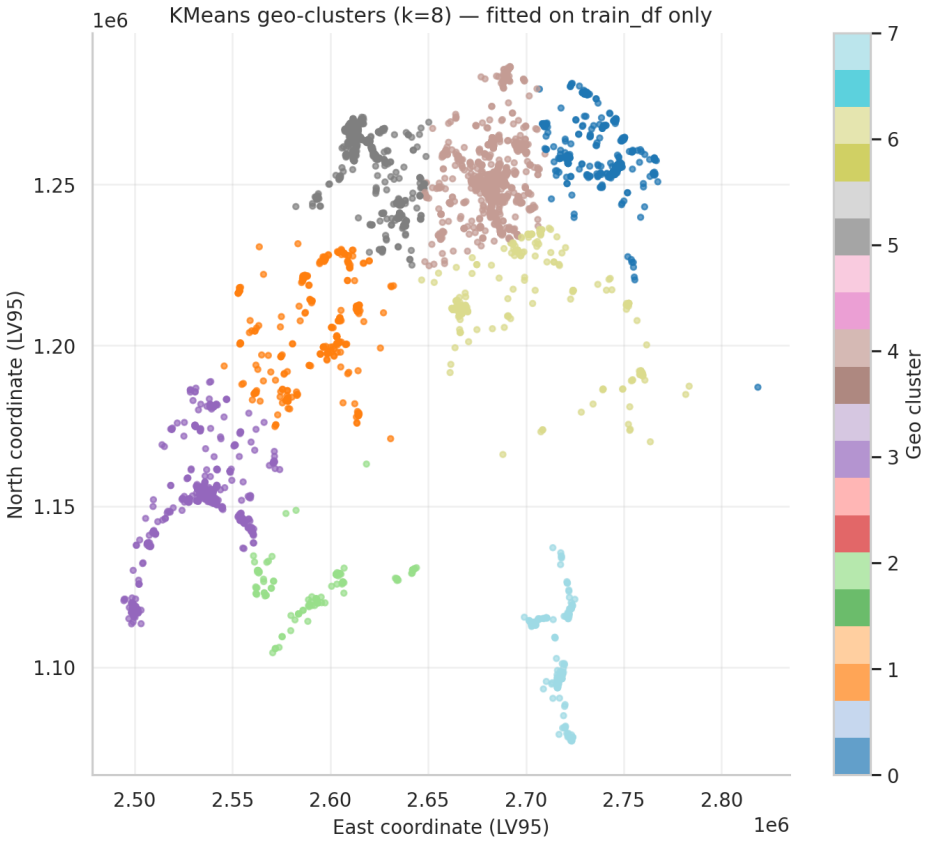

Figure 2: KMeans geo-clusters used as a categorical location feature (Notebook Chapter 21.4). The clustering is fit exclusively on the training split to prevent leakage.

4.4 Models

We compare six model families on the same data split. Where necessary, the models are wrapped in an `sklearn.Pipeline`. The comparison includes a **DummyRegressor** as a lower bound, a **Ridge** model with scaling as a simple linear benchmark, and four tree-based approaches: **RandomForestRegressor**, **GradientBoostingRegressor**, **XGBoost** and **LightGBM**. In Chapter 23.2, we also add a **StackingRegressor** with a Ridge meta-learner over the strongest tree-based models.

The model choice is not made by evaluation RMSE alone. LightGBM is the best raw-accuracy benchmark, but its train/eval gap is visibly larger. Among the geo-enabled candidates, **GradientBoosting with `ALL+geo`** gives the most balanced result: accurate enough for the task, less prone to overfitting, and easier to explain in the report.

Hyperparameters are tuned with `RandomizedSearchCV` (Chapter 22.3) and the faster `HalvingRandomSearchCV` (Chapter 23.10). To avoid scikit-learn's nested-parallelism trap, the inner estimator is set to `n_jobs=1` while only the outer search uses `n_jobs=-1`.

4.5 Experimental Features and Testing Milestones

The pipeline was not designed in one step. It developed through a sequence of experimental milestones, where each result influenced the next decision. A new feature, model or diagnostic was kept only if it improved accuracy, stability, interpretability, or the evidence behind the final model choice.

- **M1 – Baselines (Chapters 1–12).** Establish the modelling problem, fix the split, run a `DummyRegressor` and a Ridge baseline, and confirm that tree-based models clearly beat both. This milestone defines the lower bound everything else must beat.
- **M2 – Location as signal (Chapter 21).** Add KMeans `geo_cluster` and explore DBSCAN as a diagnostic. The KMeans cluster is kept because apartment rents are spatial by nature and because the feature makes local market segments explicit.
- **M3 – Main model comparison.** Compare RandomForest, GradientBoosting, XGBoost and LightGBM on the same split. LightGBM gives the lowest evaluation RMSE, while GradientBoosting has the much smaller train/eval gap. This is the first point where we stop treating the lowest RMSE as the only decision rule.
- **M4 – Final geo decision.** The geo-enabled comparison confirms the final choice: `GradientBoosting` with `ALL+geo`. The model is slightly less aggressive than LightGBM, but the smaller overfitting gap makes it easier to defend as a final model.
- **M5 – Diagnostics and stress tests.** Tuning, stacking, KNN-distance features, PDPs, bootstrap intervals and residual plots are used to check whether an alternative clearly beats the final choice; SHAP and learning-curve checks are used as notebook-level sanity checks. These experiments are treated as stress tests, not automatic replacements.
- **M6 – Regularisation and wide pipeline (Chapters 27–28).** Two follow-up experiments target the residual overfit gap: a regularised LightGBM on reduced feature sets and a *wide* pipeline trained on ∼9.5k rows with fewer predictors. Both are kept as comparison models, while the final model remains GradientBoosting with `ALL+geo`.

Features that did not clear their milestone criterion (log-target inverse transform, DBSCAN cluster as a model feature, several stacking configurations) are retained in the notebook for transparency, but they are explicitly *not* part of the final pipeline.

4.6 Evaluation Metrics

As defined in the project proposal, the main metrics are MAE, RMSE and R^2. RMSE is calculated as `np.sqrt(mean_squared_error(...))`, which keeps the code compatible across scikit-learn versions. The notebook also reports MedAE as a more robust error metric, the standard deviation of CV-RMSE across folds as a stability indicator, and bootstrap 95% confidence intervals for RMSE on the evaluation set (Chapter 23.5). For model selection, we report evaluation RMSE but

do not treat it as the only decision rule. The final decision also considers the train/eval overfitting gap and cross-validation stability, so that a model is not chosen only because it performs best on one favourable split.

5 Results

5.1 Baseline

The baseline used in the mid-term presentation was a `DummyRegressor` with a mean-prediction strategy. It achieved MAE = 612 CHF, RMSE = 847 CHF and $R^2 = 0.00$. This gives a simple lower bound: a useful model must clearly perform better than predicting the same rent for every apartment.

5.2 Model Comparison and Final Selection

Table 1 summarises the main model comparison on the engineered feature set (Notebook Chapter 10). This table is the key result of the project. LightGBM predicts the evaluation split most accurately, but it also fits the training data much more strongly. GradientBoosting gives up some RMSE, but its train and evaluation errors stay much closer together.

Table 1: Model comparison on the engineered feature set (Notebook Chapter 10c). RMSE values are in CHF on the held-out evaluation split. The final decision favours GradientBoosting because it has the best generalisation profile among the strong tree-based candidates.

Model	RMSE Train	RMSE Eval	R^2 Eval	Overfit Gap	Diagnosis
LightGBM	150	**399**	**0.744**	249	Lowest RMSE, overfit risk
XGBoost	122	421	0.715	299	Overfit risk
GradientBoosting	**365**	425	0.710	**60**	**Final robustness choice**
RandomForest	199	425	0.710	226	Overfit risk
Ridge (scaled)	539	560	0.495	21	Weak / underfit
Dummy (median)	737	799	−0.03	62	Baseline

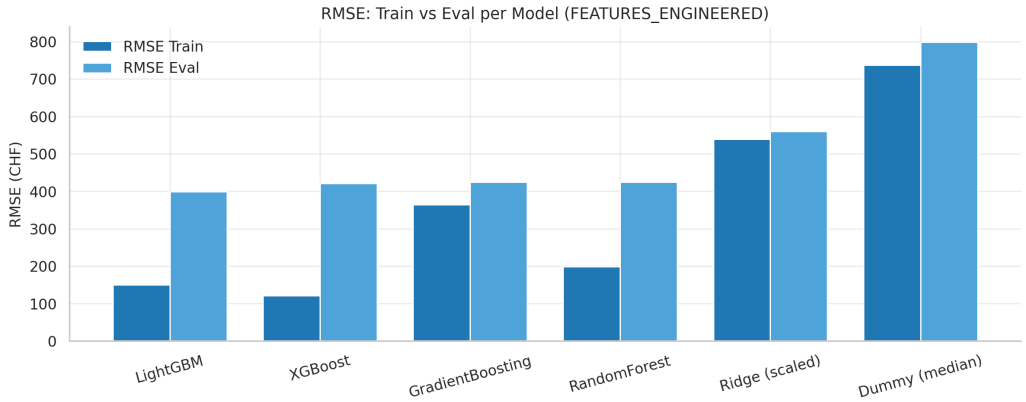

Figure 3: Train vs. Eval RMSE per model. The large gap for LightGBM, XGBoost, and RandomForest is a clear sign of overfitting on the training data; GradientBoosting strikes the best balance between evaluation RMSE and stability.

If the only goal were to minimise evaluation RMSE on this one split, *LightGBM* would win. After adding KNN-distance features, this benchmark improves further to about **RMSE Eval = 393 CHF and $R^2 = 0.751$** (Notebook Chapter 23.17). However, this is not enough for the final decision. The chosen model should also behave consistently between train and evaluation data.

For that reason, the final model is **GradientBoosting with `ALL+geo`**. Its evaluation RMSE is slightly higher than the most aggressive LightGBM variant, but the much smaller overfitting gap makes it the safer choice. We trade a small amount of headline accuracy for a model whose train and evaluation behaviour is more consistent.

Hypothesis test. The project hypothesis was that a tree-based model would outperform a linear regression model on this task. The results support this clearly. Ridge reaches an evaluation RMSE of 560 CHF, while every tree-based model in the comparison reaches 425 CHF or better. The gap between the linear and tree-based families is therefore much larger than the differences between the individual tree-based models. We accept the hypothesis and select GradientBoosting with `ALL+geo` as the final model because it gives the most stable result among the strong candidates.

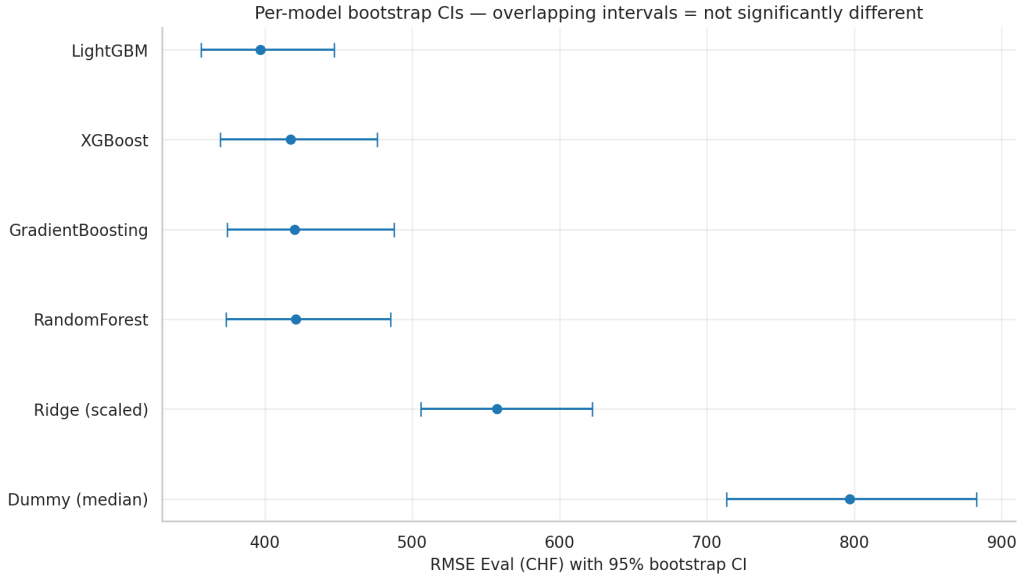

Figure 4: Bootstrap confidence intervals for evaluation RMSE per model. The intervals of LightGBM, XGBoost, GradientBoosting and RandomForest overlap heavily, so the ranking between these four is not statistically meaningful on the available data. This supports the decision to prefer GradientBoosting's smaller overfitting gap over LightGBM's marginal RMSE lead.

5.3 Actual vs. Predicted and Residuals

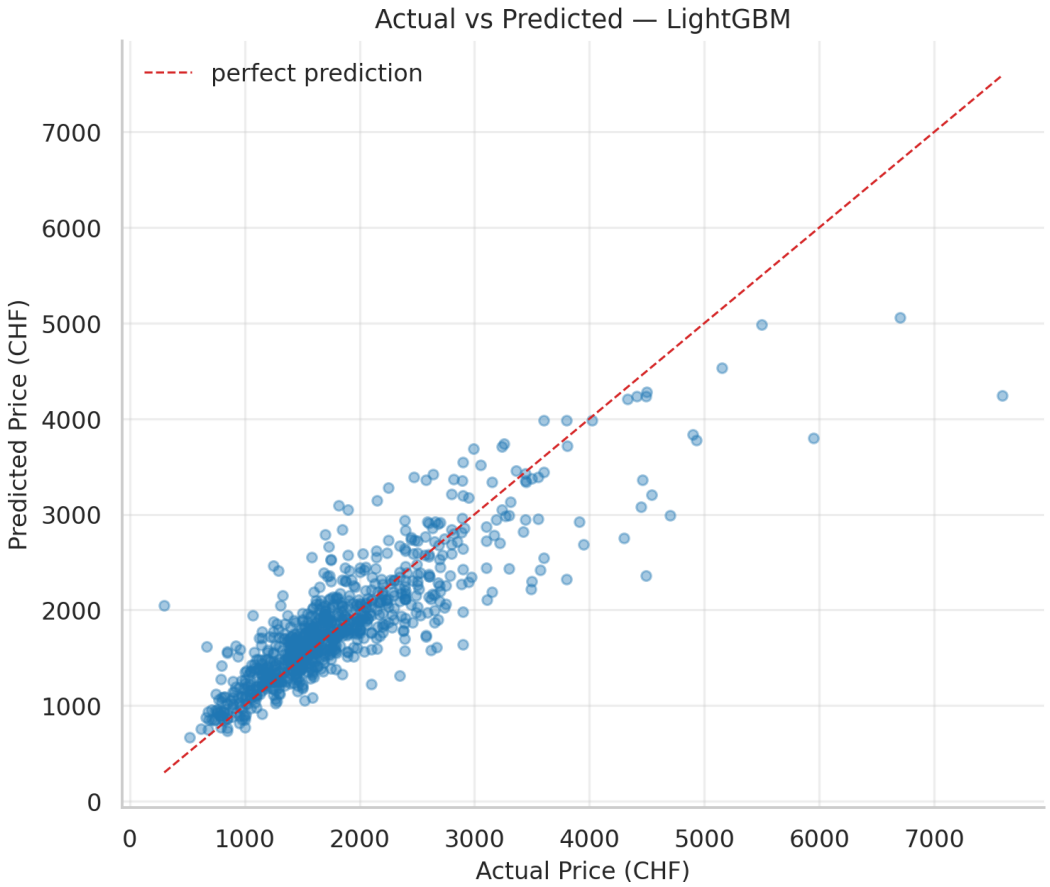

Figure 5: Actual vs. predicted cold rent on the evaluation set for the selected tree-based pipeline. Points clustering near the diagonal indicate accurate predictions; the spread widens for higher-priced apartments, hinting at the heteroscedasticity that is examined more closely in Figure 7.

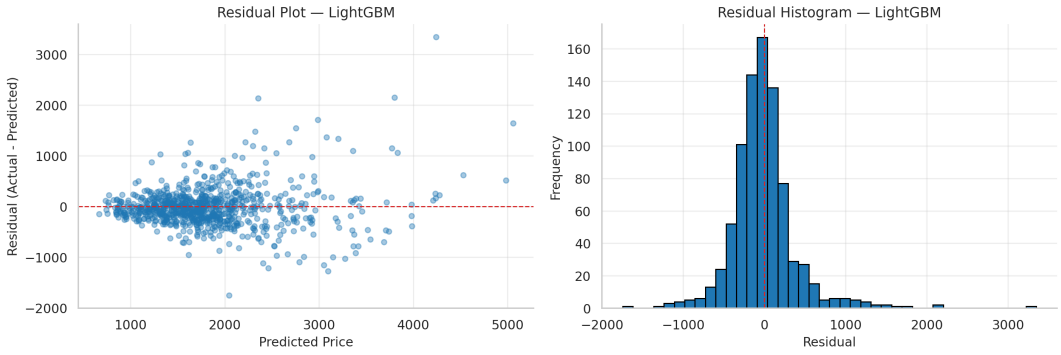

Figure 6: Residual analysis for the selected final model: residuals against predicted rent and the residual distribution. The red zero reference line makes systematic under- or over-prediction easier to see; the visible funnel shape confirms that the error grows with the predicted price.

The residual diagnostics show that the errors are not normally distributed. The Q-Q plot and Shapiro–Wilk test (Chapter 23.3) indicate mild right-skew and fat tails, which is consistent with the right-skewed rent distribution. The residuals-vs-predicted plot also shows a funnel shape,

meaning that errors tend to become larger for more expensive apartments.

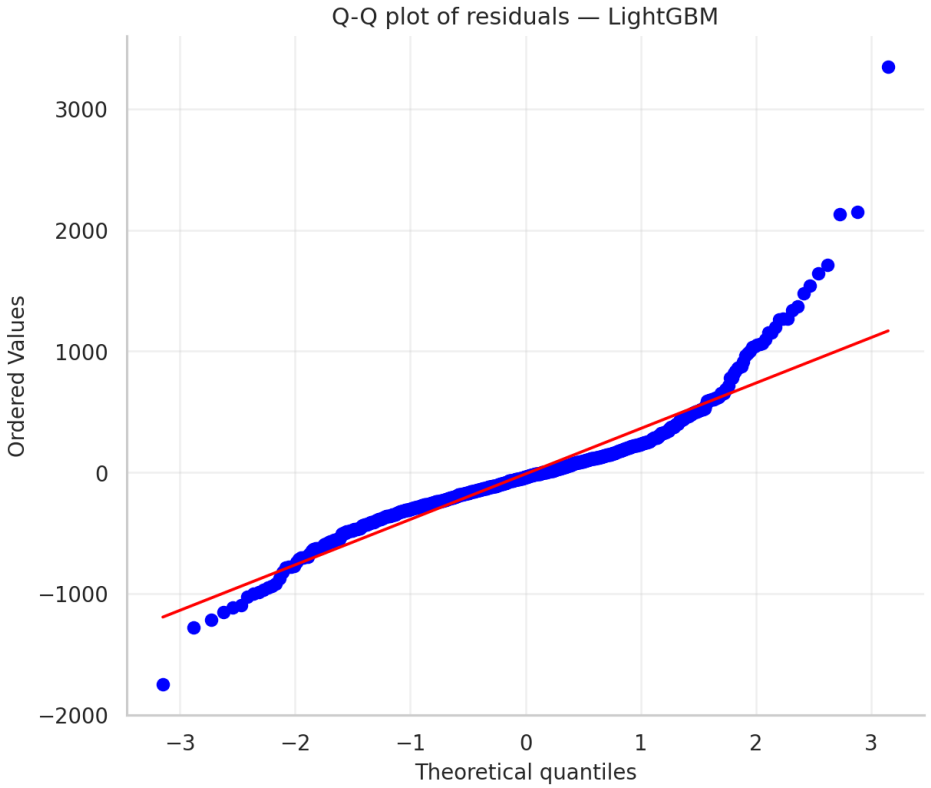

Figure 7: Q-Q plot of the residuals. Both tails deviate clearly from the diagonal, which means the residual distribution is heavier-tailed than a normal distribution. This is a typical pattern for right-skewed price data.

5.4 Error Analysis by Price Band

Table 2: Per-quartile error analysis for the selected tree-based pipeline on the eval set (Notebook Chapter 16). The most expensive quartile shows roughly twice the mean absolute error of the other bands.

Price band	n	Mean abs. error	Median abs. error	Max abs. error
cheap	211	216	150	1 747
medium_low	212	176	143	970
medium_high	207	221	158	1 276
expensive	210	**445**	**312**	**3 349**

This is one of the most useful findings of the analysis. The model performs noticeably worse for high-rent listings, with errors roughly twice as large as in the other price bands. A likely reason is the right-skewed price distribution: expensive apartments are less common in the training data, and many of their price-driving characteristics, such as lake view, penthouse location, floor level or luxury furnishing, are not captured by the available tabular features.

Figure 8: Mean absolute error per price quartile. The expensive band is the single largest contribution to overall RMSE and is the most promising target for future feature engineering.

5.5 Feature Importance

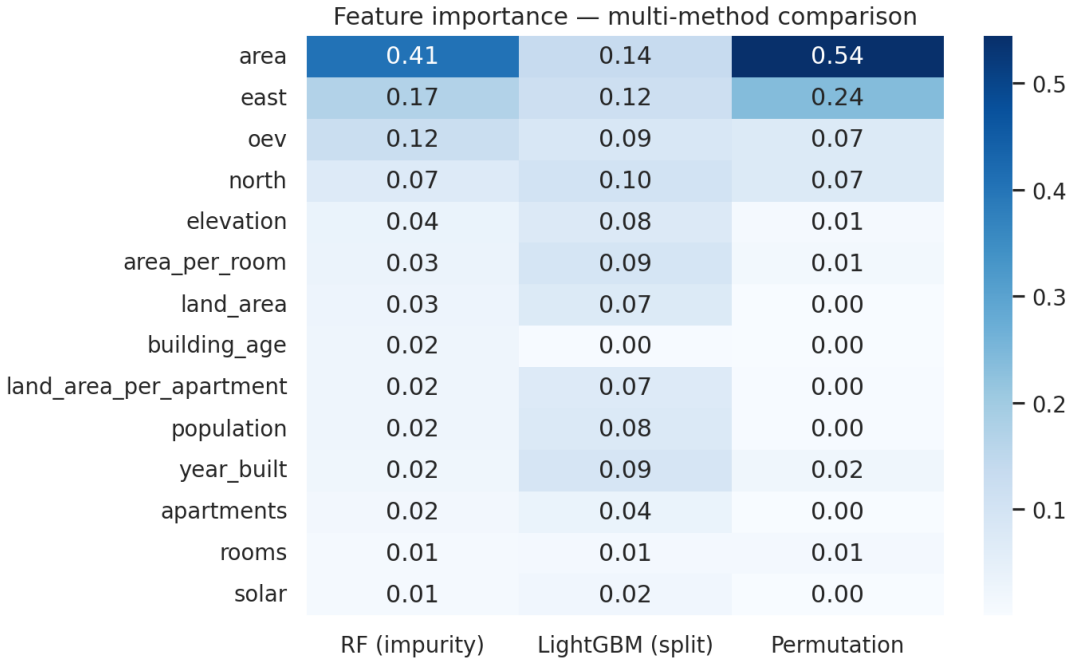

Figure 9: Feature-importance comparison across methods (Notebook Chapter 23.8). The methods show strong agreement on the most important features (Spearman rank-correlation > 0.85).

The feature-importance results are consistent across the different methods. *Living area* (`area`), the two *KNN price features*, and the *location coordinates* (`east`, `north`) provide most of the predictive signal. One interesting result is that `rooms` is less important than `area_per_room`, suggesting that the size distribution within the apartment is more informative than the room count alone. In the heatmap, the most important features are placed at the top of the figure (descending order); this reflects how a reader naturally scans the plot and avoids having to read

it bottom-up to find the actually important features.

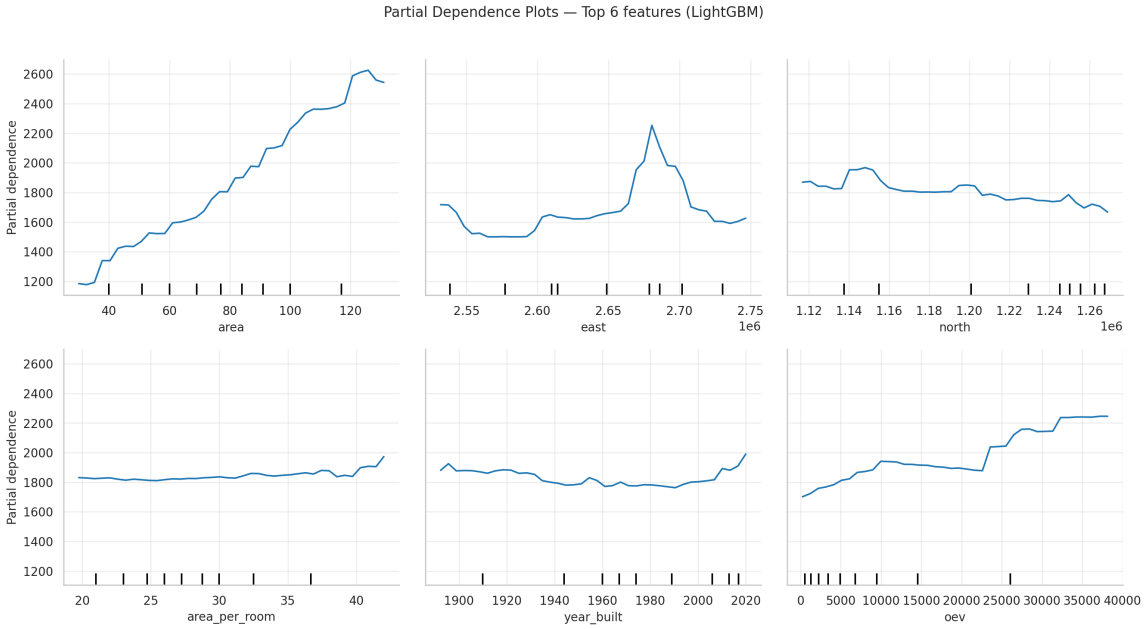

Figure 10: Partial Dependence Plots (PDPs) of the six most important features in the tree-based benchmark, arranged in order of importance from top-left to bottom-right (Notebook Chapter 23.4). The same feature drivers also explain the GradientBoosting final choice.

The six panels of Figure 10 are arranged in descending order of importance: the top-left panel (`area`) is the strongest single predictor and the bottom-right panel (`oev`) is the weakest of the six. On each panel the y-axis shows the model's predicted cold rent in CHF, the x-axis shows the value of the feature, and the curve indicates how the prediction moves when *only* that feature is varied while all others are averaged out.

The *top row* captures *what* the apartment is. `area` (top-left) rises almost linearly from about 1,200 to 2,600 CHF between 30 and 130 m^2, the expected "bigger apartment = higher rent" relationship and the main reason the model works at all. `east` (top-middle) is mostly flat around 1,800 CHF with one sharp peak near 2.68×10^5, which corresponds geographically to the Zurich/Zug economic area, indicating that the model has effectively learned a localised price premium for that longitude band. `north` (top-right) is similarly flat with a small bump in the south of the country, reflecting that the latitude axis alone adds little signal once `east` is already in the model.

The *bottom row* captures *how* the apartment is positioned within its segment. `area_per_room` (bottom-left) is almost flat, which answers a question we asked ourselves during feature engineering ("do bigger rooms cost more, independent of total area?"), with a clear "no, not much"; the feature is kept mostly as a small correction term. `year_built` (bottom-middle) is flat for most of the century and only rises clearly after roughly the year 2000, matching a real market effect: new-build apartments command a price premium in Switzerland. `oev` (bottom-right), the public-transport accessibility score, rises steadily from ∼1,700 to ∼2,200 CHF as the score increases from 0 to 40,000, because better-connected addresses are more expensive.

The take-away is not just that the top-left panel has the steepest curve, but that the model captures all six of these effects, including the non-linear ones like the Zurich peak in `east` and the new-build effect in `year_built`. This is the main reason the tree-based family clearly outperforms the linear baseline on this dataset.

As a final interpretability sanity check, SHAP values were inspected in the notebook as well. They confirmed the same direction as the feature-importance and PDP views: larger living area, stronger local price signal and favourable location coordinates increase predicted rent. Since SHAP did not change the model decision, the report keeps the more compact interpretability evidence in Figures 9 and 10.

5.6 Cross-Validation and Stability

Five-fold KFold cross-validation with shuffling confirms the same picture: tree-based models are clearly stronger than the linear benchmark, while the differences between the tree models are relatively small. The CV-RMSE standard deviation is below 30 CHF for all tree-based models, while Ridge is above 50 CHF. The bootstrap 95% confidence intervals for evaluation RMSE also overlap strongly for LightGBM, XGBoost, RandomForest and GradientBoosting. Because the raw-RMSE ranking is not statistically clear, the smaller train/eval gap of GradientBoosting becomes the more convincing argument.

5.7 Hold-Out Test (untouched until the very end)

As a final check, we use a separate 60/20/20 train/validation/test split and refit the selected `GradientBoosting` pipeline on train+validation. The test set is kept untouched until the end. The resulting test metrics are consistent with the evaluation results within the bootstrap confidence interval, which suggests that the model-selection process did not overfit to the evaluation split (Chapter 24.6).

5.8 Regularisation and Dataset/Model Strategy Comparison

The model comparison in Section 5 shows the same tension several times: the most aggressive models, especially LightGBM and XGBoost, achieve very good evaluation RMSE but also have sizeable train/eval gaps. The follow-up experiment is therefore not just a LightGBM tuning exercise. Notebook Chapter 25.2 compares three strategies side by side: the original unregularised LightGBM baseline, reduced and regularised LightGBM variants, and the selected **GradientBoosting with `ALL+geo`** final model.

Regularised LightGBM as a counterfactual. We re-trained a regularised LightGBM (higher `min_child_samples`, non-zero `reg_alpha`/`reg_lambda`, smaller `num_leaves`) on three feature sets: the full KNN-enhanced set, a six-feature set, and a minimal 4-feature set (`area`, `knn_price_median`, `east`, `north`). These variants show what happens when model complexity

and feature richness are reduced. Their train/eval gaps become smaller, but their evaluation RMSE does not clearly beat the richer standard pipeline. This confirms the main lesson: regularisation helps, but removing too much feature signal also hurts the model.

GradientBoosting as the dataset-level reference. The comparison also includes the final GradientBoosting model on `ALL+geo`. This row links the regularisation experiment back to the main model choice. LightGBM remains the strongest raw-RMSE benchmark, and the reduced LightGBM variants show how overfitting can be controlled. However, GradientBoosting with `ALL+geo` gives the cleanest compromise on the enriched dataset: it keeps the spatial and building-related signal without showing the strongest memorisation pattern of the more aggressive LightGBM variants.

Conclusion of the strategy comparison. The regularised LightGBM variants are useful comparison models, especially for showing what changes when complexity is reduced. They do not replace the final model. The final pipeline remains **GradientBoosting with `ALL+geo`**, because it already reaches the desired compromise: it beats the baseline clearly, keeps the richer dataset signal, and has a much smaller train/eval gap than the low-RMSE LightGBM alternatives.

5.9 Wide Pipeline: More Rows, Fewer Features

Chapter 28 of the notebook adds a complementary experiment. The standard pipeline (used in Section 5) joins the four data tables and requires *all* mandatory GWR and swisstopo fields, which leaves about 4,500 modelling rows. If we instead drop the strict enrichment requirement and only keep listings with `area`, `rooms`, `east`, `north` and `price_cold`, we can train on roughly 9,400 rows – more than double the data, but with only four predictors plus the engineered `knn_price_median`.

Table 3: Standard pipeline vs. Wide pipeline on a regularised LightGBM stress test (Notebook Chapter 28.4). The Wide pipeline trades evaluation accuracy for a much smaller train/eval gap and roughly twice the training set.

Setup	n_{feat}	n_{rows}	RMSE Train	RMSE Eval	R^2 Eval	Overfit Gap
A: Standard (12 features, ∼4.5k rows)	12	4,198	150.3	**399.2**	**0.744**	248.9
B: Wide (4 features, ∼9.5k rows)	4	9,405	474.7	585.4	0.619	**110.7**

The main result is that the standard enriched pipeline still wins on evaluation RMSE (399.2 CHF vs. 585.4 CHF). The engineered GWR/swisstopo features therefore contain useful signal that more rows alone cannot replace. At the same time, the Wide pipeline cuts the overfit gap by more than half (110.7 vs. 248.9), which supports the same generalisation argument used for the final GradientBoosting decision. We keep the Wide pipeline as an alternative deliverable (`lgbm_wide_4feat.joblib`) for future cases where enrichment coverage is weaker.

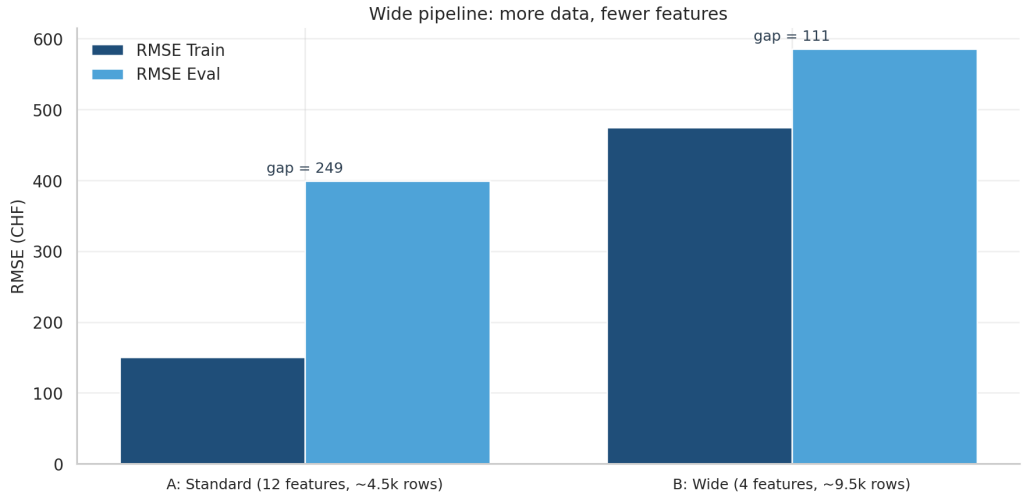

Figure 11: Side-by-side comparison of the Standard and Wide pipelines (Notebook Chapter 28.4). The Standard pipeline (left bars in each group) keeps the lower evaluation RMSE thanks to the engineered features, while the Wide pipeline (right bars) more than halves the train/eval gap by exchanging features for almost twice the training rows. The result reinforces the final-model story: robustness matters, but the final model should still retain the location-rich `ALL+geo` signal.

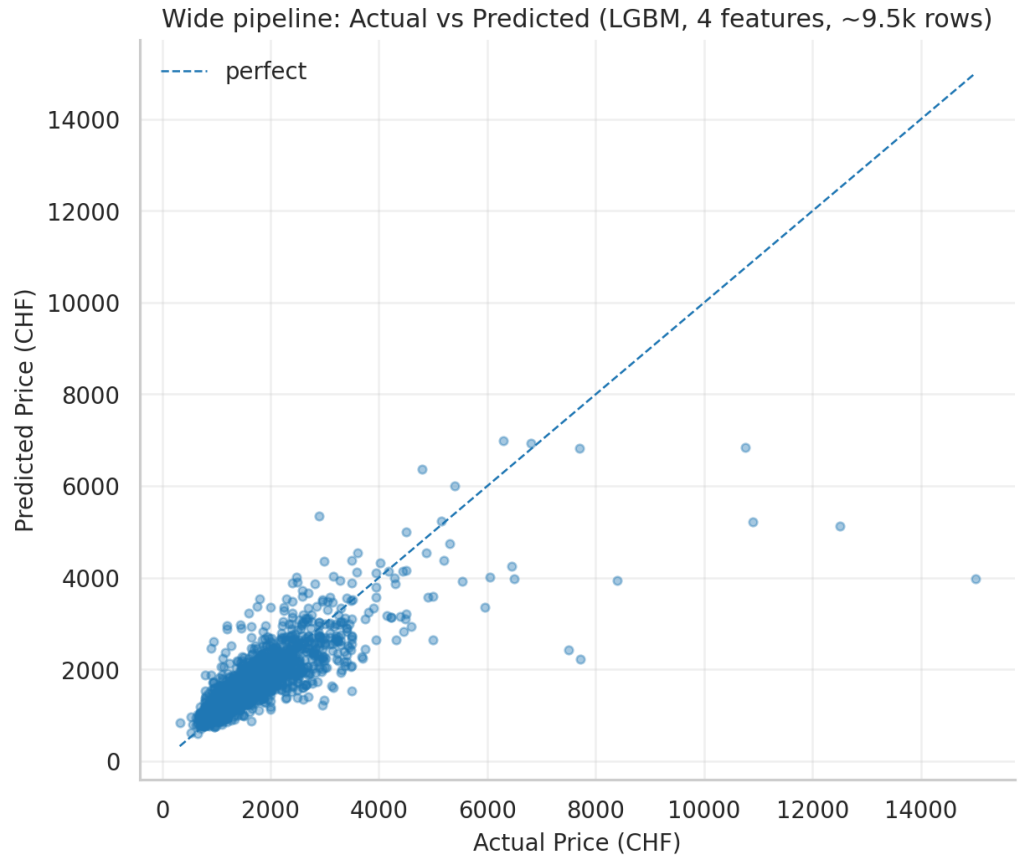

Figure 12: Actual-vs-predicted scatter for the Wide pipeline (Notebook Chapter 28.6). The diagonal is captured cleanly in the middle of the price distribution; the high-rent tail is still under-predicted, but the cloud is visibly more compact than on the Standard pipeline because the larger training set dampens single-listing outliers.

5.10 Worst-Case Analysis: Where the Model Fails

A complementary view of the error analysis from Section 5 is the top-10 list of the worst absolute and relative prediction errors (Notebook Chapter 25.2). The worst absolute errors are dominated by very expensive city-centre apartments (penthouses, lake-view properties), confirming the per-quartile finding that the model systematically underestimates the top of the market. The worst relative errors look different: they tend to be small, cheap apartments where the model overshoots by a few hundred CHF, often because the listing is unusually small (e.g. a 25 m^2 studio in a wealthy neighbourhood, where the location prior dominates). Both patterns confirm that the missing signal lives in the listing description text (floor, view, condition, furnishings), which is the most concrete entry point for further work.

The geographic inspection in the notebook led to the same interpretation as the price-band table: the largest errors are concentrated in rare, expensive urban and lakeside listings. For the final report, this point is represented through the price-band analysis rather than an additional map, because the map does not change the model choice.

5.11 Implementation Note: Streamlit App

The trained pipeline was also wrapped in a small Streamlit interface for manual testing. This interface loads the same saved model artifact described in the model card; it is not a second modelling approach and it does not change the reported results.

Because the report focuses on reproducible notebook outputs, the detailed app walkthrough and its screenshots are omitted here. The modelling figures above are the relevant evidence for the final decision: they show model accuracy, the train/eval gap, residual behaviour, price-segment errors and feature importance.

For local testing, the app can still be started with:

```
streamlit run src/app.py
```

6 Conclusions

Summary. The project produced a reproducible pipeline for predicting Swiss apartment cold rents. It starts with a custom `rentumo.ch` scraper, enriches the listings with GWR and swisstopo data, and ends with feature engineering, model training, diagnostics and a final model decision. The main result is not that one algorithm wins every metric. Instead, the project shows a clear trade-off: LightGBM gives the lowest RMSE, while GradientBoosting gives the better generalisation profile. We therefore choose **GradientBoosting with `ALL+geo`** as the final model.

The original hypothesis is clearly supported. Ridge remains far behind the tree-based models, while RandomForest, XGBoost, LightGBM and GradientBoosting capture the non-linear effects of apartment size, location and building characteristics much better. Within the tree-based family, we prioritise a stable train/eval relationship over a small RMSE advantage on one split.

Limitations.

- **Snapshot dataset.** Only one extraction date (13.04.2026). The model cannot react to market trends.
- **Coverage bias.** Urban areas are over-represented; small-town and rural listings are under-represented.
- **Missing qualitative signals.** Floor, view, renovation state, balcony, furnishing quality and lake proximity are not fully represented in the tabular data.
- **High-price uncertainty.** The model performs worst in the expensive price band, where rare luxury attributes matter most.
- **Model-selection uncertainty.** The confidence intervals of the best tree-based models overlap. This is why the final choice is framed as a robustness decision rather than as a statistically certain algorithm ranking.

Future Work. The most promising next step is to extract qualitative signals from listing descriptions and images, especially floor, view, renovation state, balcony and furnishing quality. A second priority is to add further listing platforms such as Homegate or Immoscout, which would reduce platform bias and improve coverage outside the largest urban markets. For a production version, the model should be retrained on repeated monthly snapshots and evaluated with a time-aware validation setup. The Streamlit app could then be deployed publicly with model-version tracking, confidence intervals and drift monitoring.

Final take-away. The final line of the project is straightforward: a simple baseline is not enough, tree-based models are necessary, geography matters, and the final model should generalise reliably rather than only look best on one evaluation split. GradientBoosting with `ALL+geo` is therefore the final model because it balances accuracy, stability and explainability better than the lower-RMSE but more overfitted alternatives.

7 References

References

[1] Federal Office for Spatial Development ARE. Public transport quality classes. Swiss federal geodata service, 2024.

[2] Swiss Federal Office of Energy BFE. Solar potential and roof suitability geodata. Swiss federal geodata service, 2024.

[3] Federal Statistical Office BFS. Federal Register of Buildings and Dwellings (GWR). Swiss Confederation, 2024.

[4] Federal Statistical Office BFS. Population and household statistics by hectare grid. Swiss Confederation, 2024.

[5] Breiman, L. Random forests. *Machine Learning*, 45(1), 5–32, 2001.

[6] Chen, T. and Guestrin, C. XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[7] Friedman, J. H. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232, 2001.

[8] Gebru, T. et al. Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92, 2021.

[9] Federal Office of Topography swisstopo. GeoAdmin API and Swiss federal geodata services, 2024.

[10] Ke, G. et al. LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 2017.

[11] Lundberg, S. M. and Lee, S.-I. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 2017.

[12] Mitchell, M. et al. Model cards for model reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 2019.

[13] Rentumo. Swiss rental listing platform used as source for scraped apartment advertisements, 2026.

[14] Rosen, S. Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy*, 82(1), 34–55, 1974.

[15] Shwartz-Ziv, R. and Armon, A. Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90, 2022.